Penjelasan Visual End-to-End Sistem Pertamina GLD

Versi Dokumen: 1.0.0

Tanggal Pembuatan: 2026-06-29

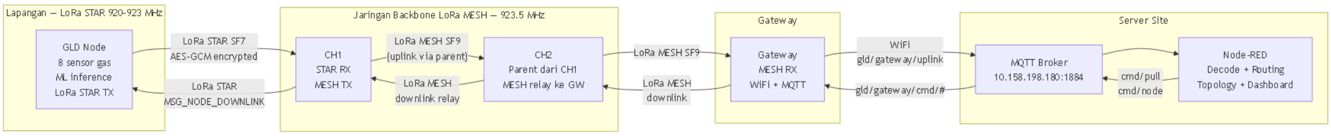
Source Basis Repo: `firmware/platformio.ini`, `firmware/shared/include/*.h`, `firmware/config/*.h`, `firmware/gld/`, `firmware/ch/`, `firmware/gateway/`, `docs/design/*/final_design.md`, `server/nodered/`

Status Firmware: GLD `0.8.12` · CH `0.7.1` · Gateway `0.1.3` · Protocol `0.1.0`

Daftar Isi

1. Visual Executive Summary

Alur Data GLD ke Server (dan Downlink Server ke GLD)



Gambar 1: Alur data GLD ke Server (uplink) dan Server ke GLD (downlink). CH tersusun parent-child — semua traffic MESH melewati jalur parent yang sama.

Tabel Ringkasan Komponen

Komponen	Perangkat	Firmware	Peran Utama
GLD	ESP32-WROOM-1U	`0.8.12`	Sensor gas, ML inference, enkripsi, LoRa STAR TX
CH1	ESP32-WROOM-1U	`0.7.1`	STAR RX (dari GLD), NodeCache, MESH TX ke parent
CH2	ESP32-WROOM-1U	`0.7.1`	Parent dari CH1, MESH relay ke Gateway
Gateway	ESP32-WROOM-1U	`0.1.3`	MESH RX, MQTT bridge ke Server Site
Server Dataset	PC/Server	—	MQTT Broker Dataset + Node-RED untuk capture data training
Server Site	PC/Server	—	MQTT Broker Site + Node-RED untuk monitoring, alarm, topologi

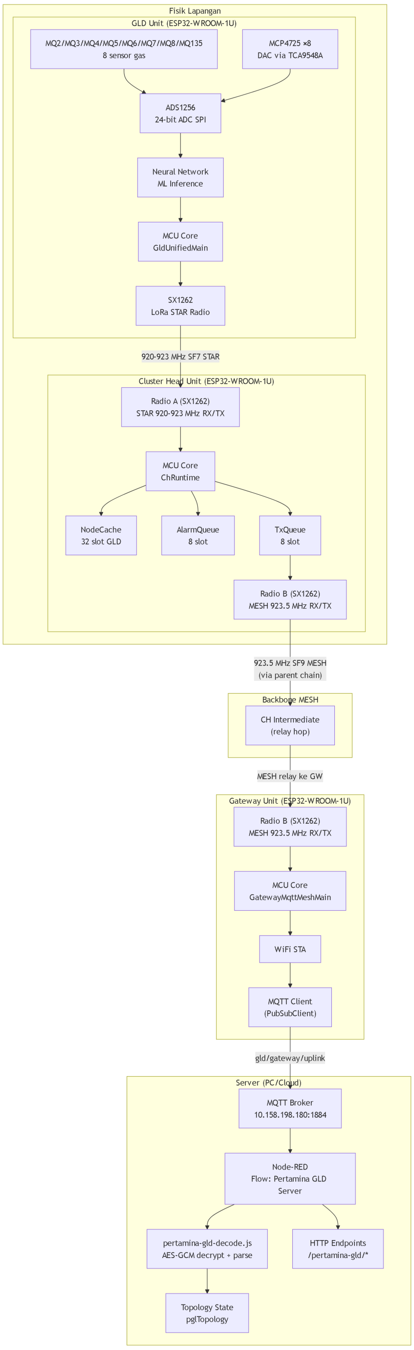
Ringkasan Alur Data Utama

Alur	Trigger	Jalur Lengkap
Normal Cache Update	Tiap 10 detik	GLD ke CH (simpan di NodeCache, tidak diteruskan otomatis ke server)
Server Pull Data	Node-RED request	Node-RED ke MQTT ke GW ke CH (hopList relay) ke GW ke MQTT ke Node-RED
Alarm Push	GasClass berbeda 0 dan conf lebih dari sama dengan 30	GLD ke CH ke parent chain CH ke GW ke MQTT ke Node-RED
Dataset Capture	Mode dataset aktif	GLD ke MQTT Broker Dataset ke Node-RED
Downlink Command	Operator	Node-RED ke MQTT ke GW ke CH ke GLD (via LoRa STAR)
Topology Report	CH boot + tiap 5 menit	CH ke parent CHx ke GW ke MQTT ke Node-RED
Nulling	Mode nulling, offline	GLD lokal saja (DAC/ADS), tidak ke server

■ ***Penting:*** *Data sensor GLD tidak otomatis diteruskan ke server. CH hanya menyimpan di NodeCache. Data dikirim ke server hanya jika Node-RED mengirim pull request ke CH tersebut.*

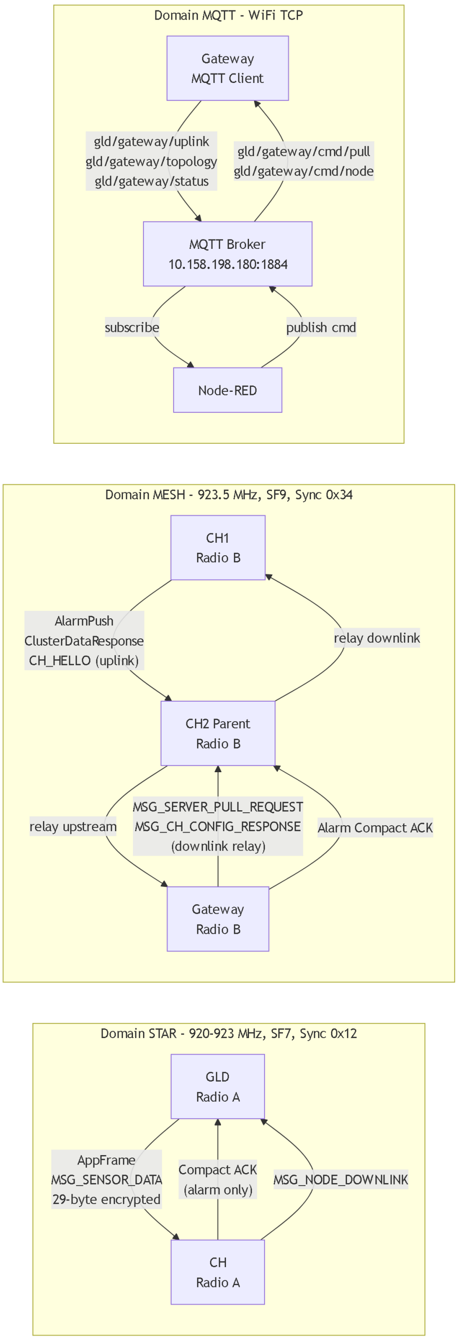
2. Peta Arsitektur Sistem

Diagram Arsitektur End-to-End



Gambar 2: Arsitektur end-to-end lengkap dengan komponen internal setiap node.

Diagram Jalur Komunikasi



Gambar 3: Tiga domain komunikasi: STAR (GLD ke CH, kiri), MESH (CH parent-child ke GW, tengah), MQTT (GW ke Server, kanan).

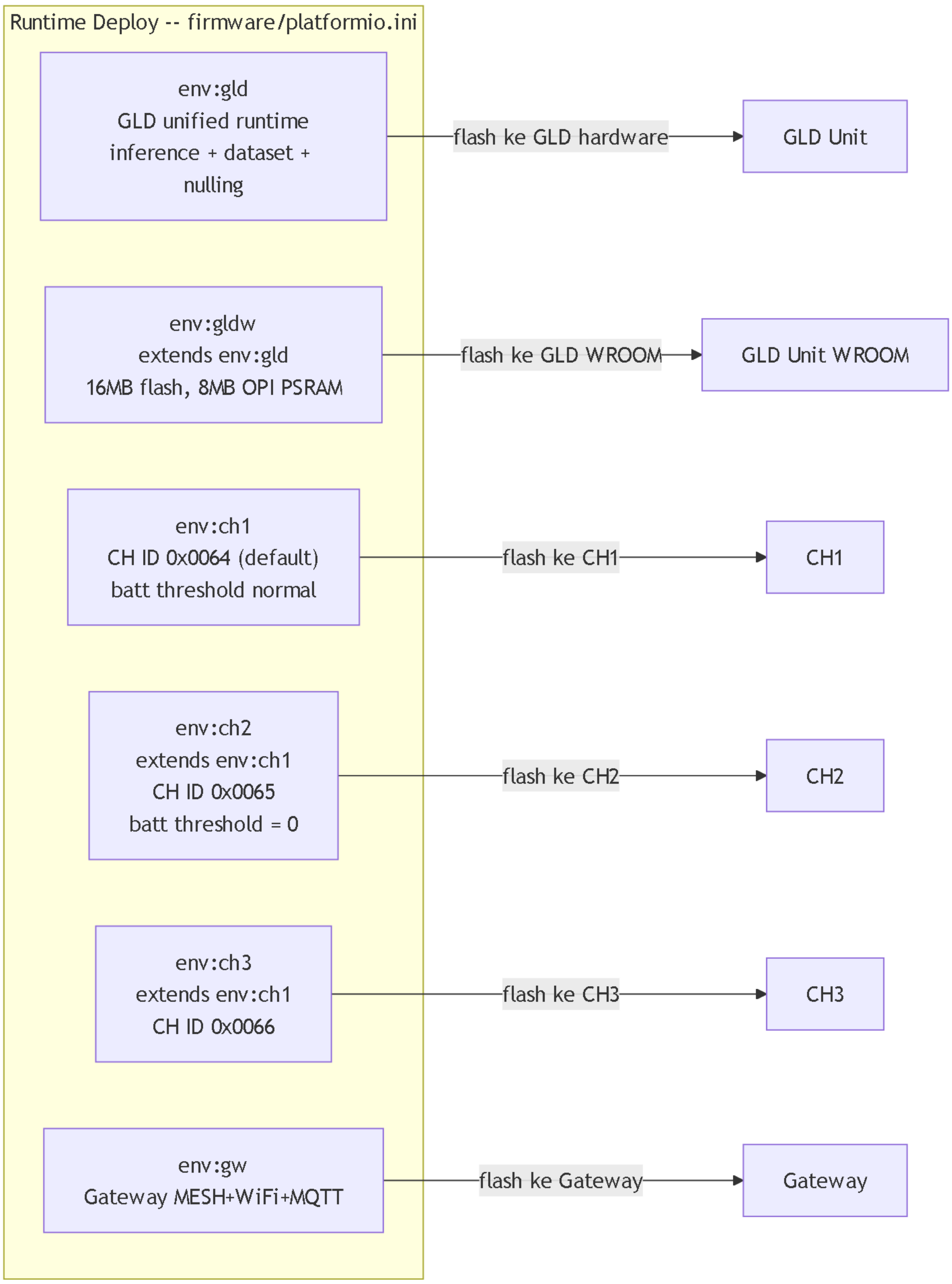
Penjelasan Per Jalur

Jalur	Frekuensi	SF	Sync Word	Max Payload	Arah
LoRa STAR	920-923 MHz (per-CH berbeda)	SF7	`0x12`	64 bytes	GLD ke CH (uplink), CH ke GLD (downlink)
LoRa MESH	923.5 MHz	SF9	`0x34`	80 bytes	CH ke CH ke Gateway (bidirectional)
MQTT/LAN	WiFi TCP	—	—	1024 bytes	Gateway ke Server

■ ***Catatan:*** *Frekuensi deployment di atas adalah nilai operasional. Source config saat ini menggunakan 920.0 MHz (STAR) dan 921.0 MHz (MESH) — perlu diupdate sesuai deployment. Dua domain LoRa menggunakan sync word berbeda agar tidak saling mengganggu. CH memiliki dua radio fisik terpisah (Radio A = STAR, Radio B = MESH).*

3. Peta Firmware Environment

Visual Map PlatformIO



Gambar 4: Peta environment PlatformIO — 6 runtime environment deploy ke hardware.

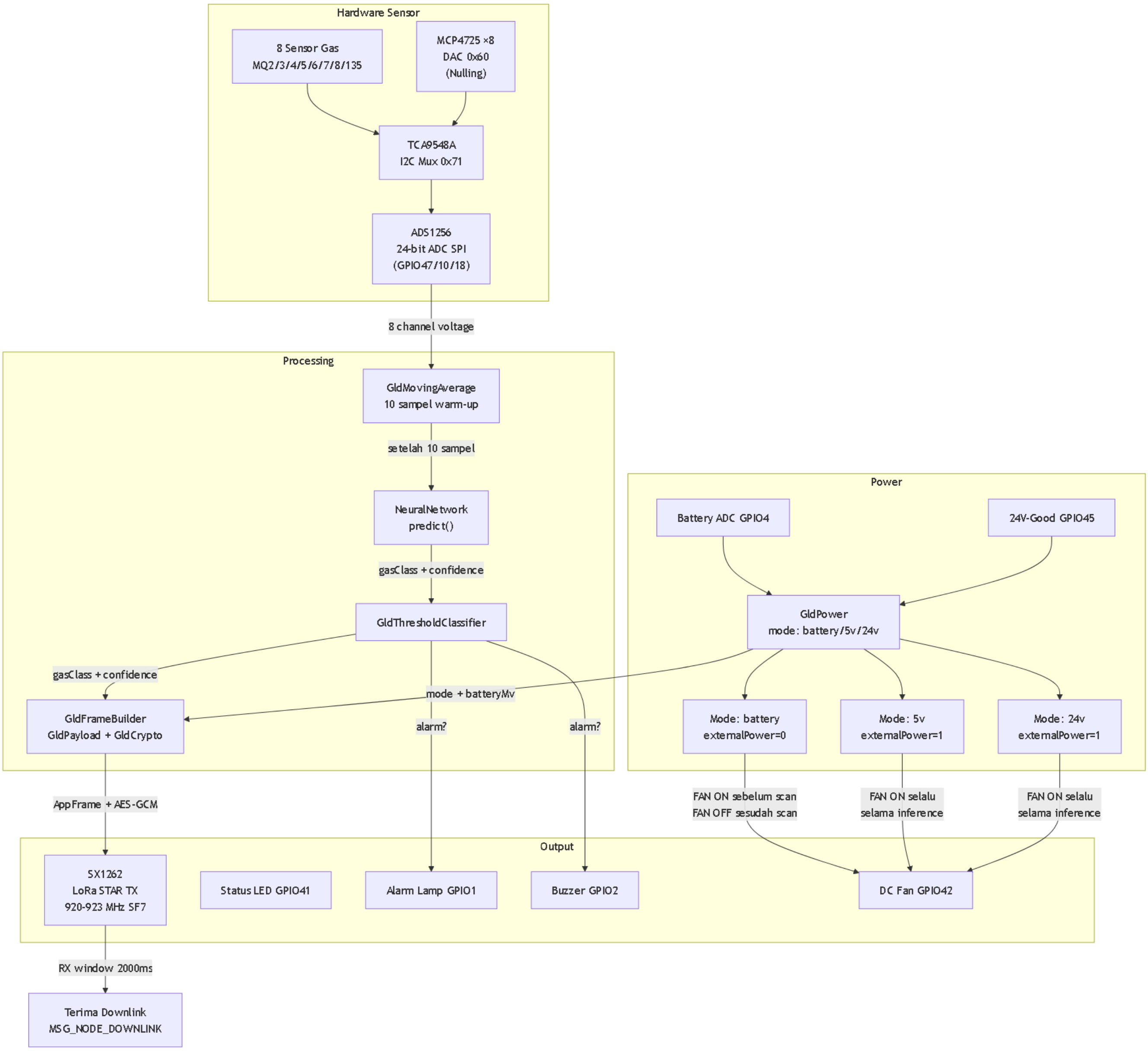
Tabel Fungsi Environment

Env	Role	Fungsi
`gld`	GLD (ID 0xF001)	GLD unified: inference + dataset + nulling
`gldw`	GLD (ID 0xF001)	GLD unified, 16MB flash, 8MB PSRAM (WROOM variant)
`ch1`	CH (ID 0x0064)	CH runtime default, batt threshold normal
`ch2`	CH (ID 0x0065)	CH runtime, batt threshold = 0
`ch3`	CH (ID 0x0066)	CH runtime
`gw`	Gateway (ID 0x006F)	Gateway MESH+WiFi+MQTT bridge ke server

■ ***Penting:*** *6 environment di atas adalah satu-satunya yang digunakan untuk deploy ke hardware (`firmware/platformio.ini`). Environment bench/selftest dan support/legacy ada di folder terpisah dan bukan untuk produksi.*

4. GLD Node

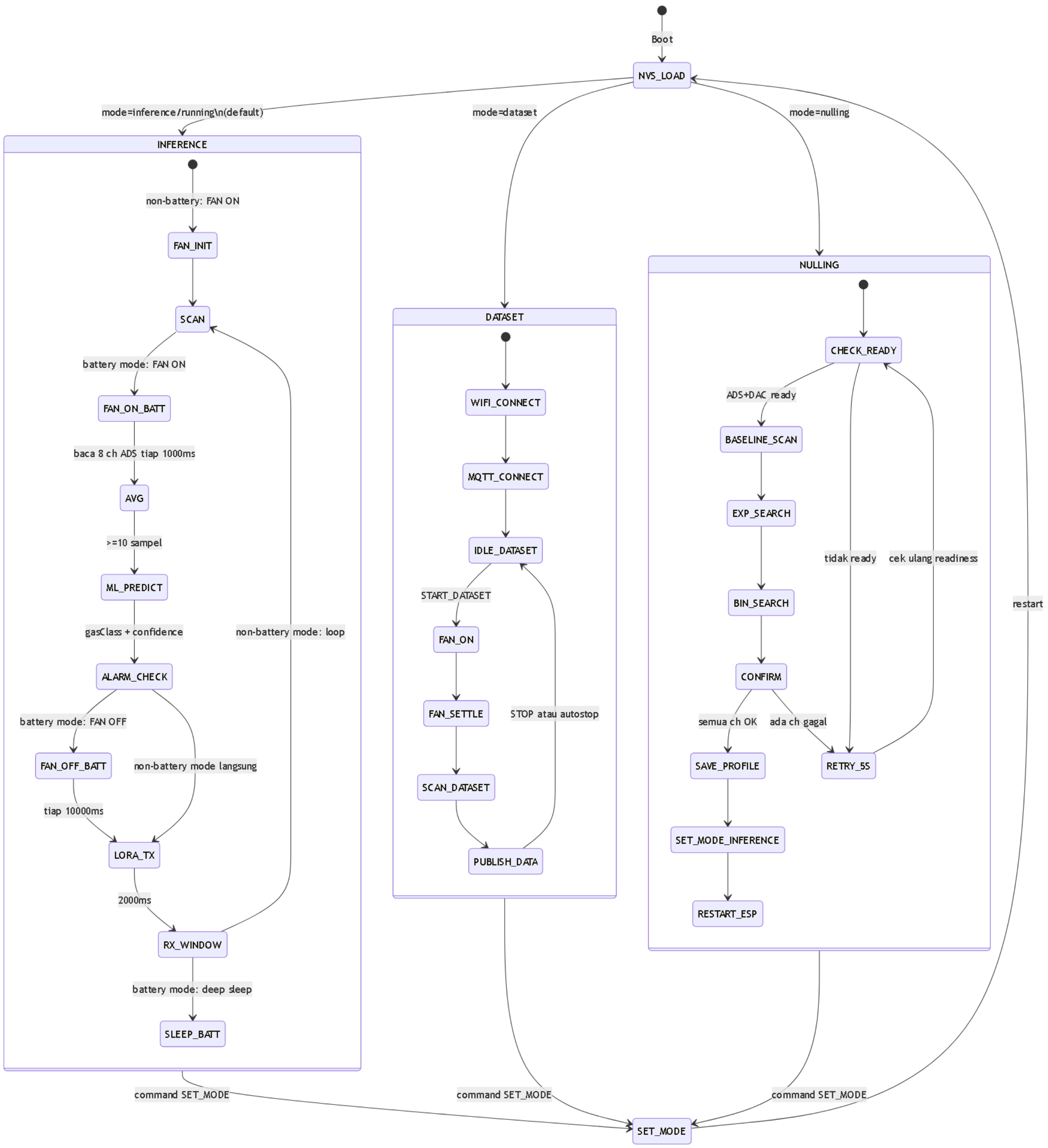
Diagram Internal GLD



Gambar 5: Diagram internal GLD Node — sensor ke frame LoRa, termasuk kontrol DC Fan dan 3 mode power.

■ ***Catatan:*** *DC Fan (GPIO42) pada inference mode adalah design intent. Firmware saat ini belum mengimplementasikan kontrol fan di inference loop — perlu firmware update.*

Diagram Mode GLD



Gambar 6: Diagram mode GLD — inference (dengan fan dan sleep battery), dataset, dan nulling.

■ ***Catatan design intent:*** *Sleep setelah RX_WINDOW pada mode battery dan kontrol DC Fan di inference loop adalah design intent yang belum diimplementasikan di firmware saat ini — keduanya perlu firmware update.*

Tabel Pin/Config Penting GLD

Fungsi	Pin/Nilai
SPI SCK/MOSI/MISO	GPIO12/GPIO11/GPIO13
ADS1256 CS/DRDY/SYNC	GPIO47/GPIO10/GPIO18
LoRa CS/RST/BUSY/DIO1	GPIO15/GPIO39/GPIO7/GPIO40 (4D)
LoRa RXEN/TXEN	GPIO5/GPIO6

Fungsi	Pin/Nilai
I2C SDA/SCL	GPIO8/GPIO9
TCA9548A	`I2C 0x71`
MCP4725 (DAC)	`I2C 0x60, range 0–4095`
Status LED	GPIO41
Alarm Lamp	GPIO1 (4D) / disabled WROOM
Buzzer	GPIO2 (4D) / disabled WROOM
DC Fan	GPIO42
Battery ADC	GPIO4
24V Power-Good	GPIO45
TPL5110 DONE	GPIO14 (OUTPUT LOW saat init)

WROOM profile overrides: LoRa CS/RST/BUSY/DIO1 → GPIO7/GPIO2/GPIO15/GPIO1; Alarm Lamp/Buzzer → disabled (`-1`).

Tabel Sensor Order dan MUX

HW Channel	Sensor	TCA Mux Ch	Model Input
0	MQ8	7	0
1	MQ135	6	2
2	MQ3	5	5
3	MQ5	4	3
4	MQ4	3	4
5	MQ7	2	6
6	MQ6	0	1
7	MQ2	1	7

Tabel Command GLD

Command	Parameter	Serial (CDC/UART0)	MQTT	LoRa Downlink
`SET_MODE inference`	mode=0	Ya	Ya (topic /cmd)	Ya (byte[0]=0x01, byte[1]=0)
`SET_MODE dataset`	mode=1	Ya	Ya (topic /cmd)	Ya (byte[0]=0x01, byte[1]=1)
`SET_MODE nulling`	mode=2	Ya	Ya (topic /cmd)	Ya (byte[0]=0x01, byte[1]=2)
`SET_MODE running`	alias inference	Ya	Ya (topic /cmd)	Tidak
`DEBUG_ON`	—	Ya	Tidak	Tidak
`DEBUG_OFF`	—	Ya	Tidak	Tidak
`START_DATASET`	label, target, dll	Tidak	Ya (topic /dataset)	Tidak
`STOP_DATASET`	—	Tidak	Ya (topic /dataset)	Tidak

MQTT topic: `gas-leak-detector/F001/cmd` untuk mode; `gas-leak-detector/F001/dataset` untuk dataset.

LoRa Downlink via `MSG_NODE_DOWNLINK`: dikirim dari CH ke GLD setelah CMD diterima dari Server.

Model ML dan Alarm Rule

Alarm rule (dari source):

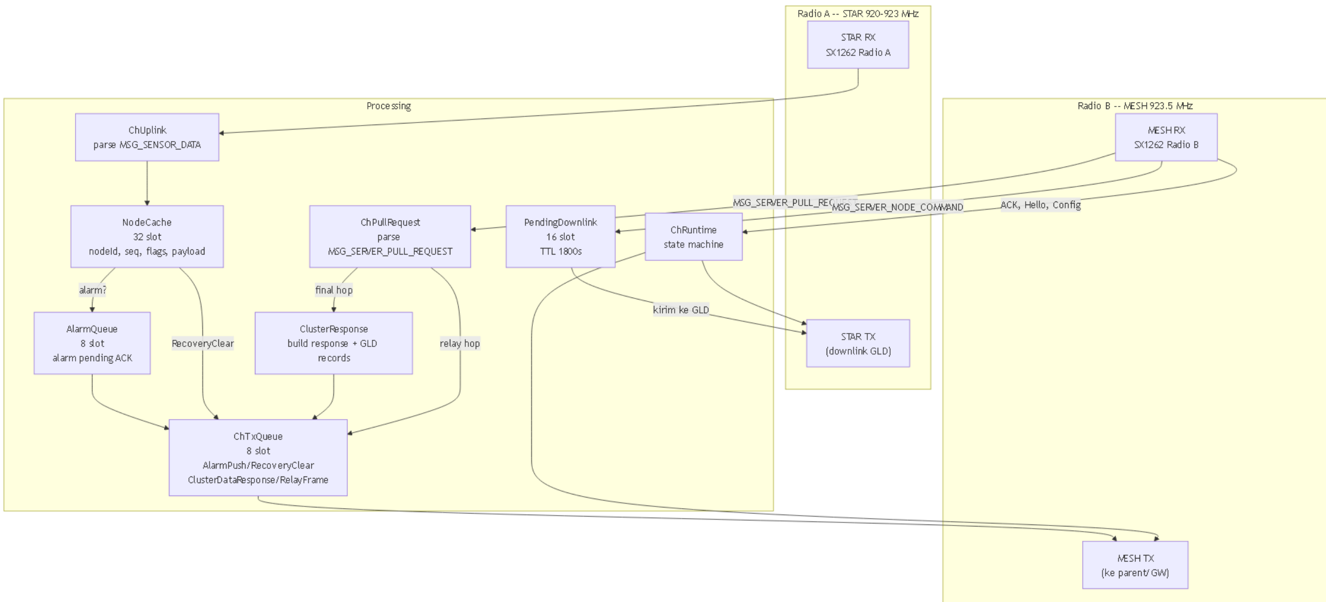
```
alarm = gasClass != clearGas (0) && confidence >= 30
```

Model Class	Protocol Gas Class	Nama Gas
0	0	clearGas
1	1	LPG
2	4	methane
3	2	propane
4	3	butane
lainnya	6	anomaly

■ ***Catatan:*** *Confidence disimpan sebagai* `uint8_t(confidenceFloat * 100.0f)`*. Nilai valid 0–100.*

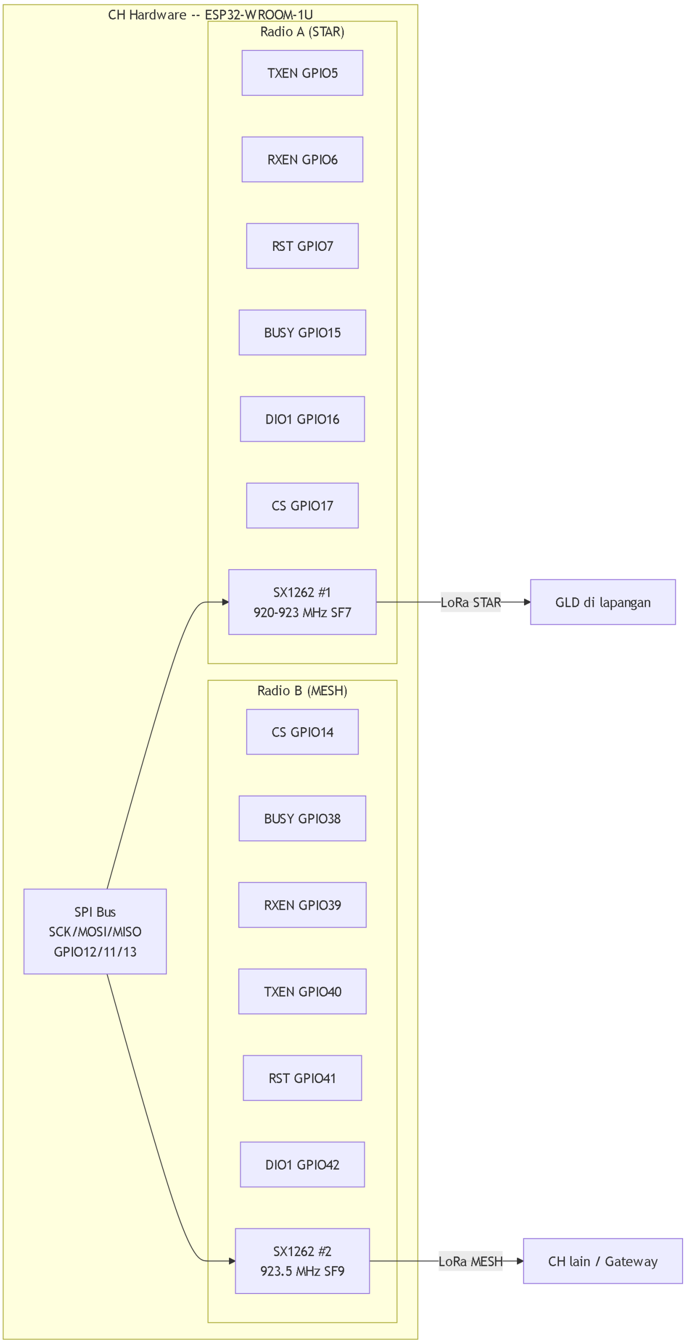
5. Cluster Head

Diagram Internal CH



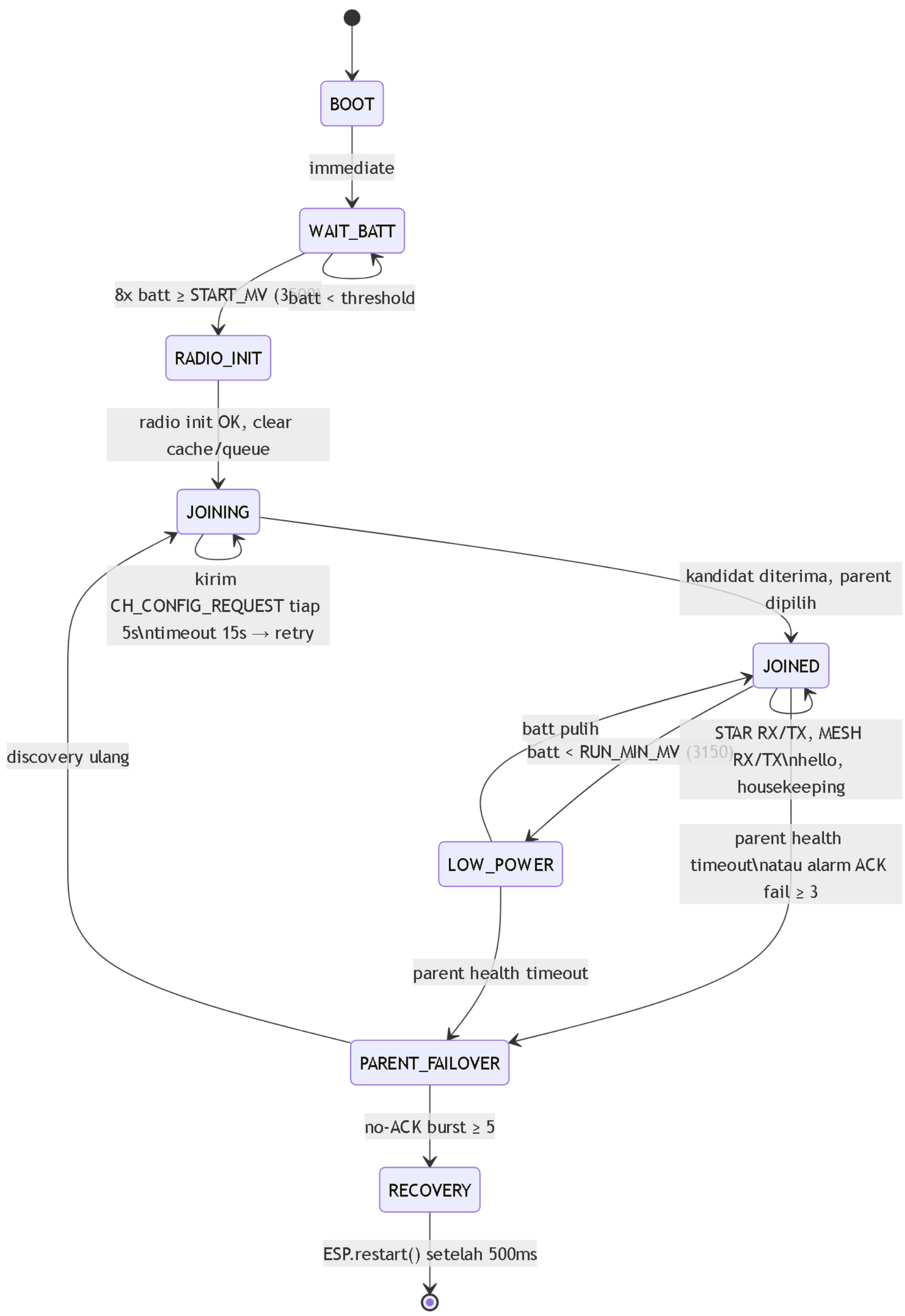
Gambar 7: Diagram internal Cluster Head — dual radio, cache, queue, dan routing.

Diagram Dual Radio CH



Gambar 8: Dual radio CH — Radio A untuk STAR (GLD), Radio B untuk MESH (backbone).

Diagram State Machine CH



Gambar 9: State machine Cluster Head — dari boot hingga operasi normal.

Tabel CH Identitas per Environment

Env	CH ID	Hex	Battery Thresholds
`ch1`	100	`0x0064`	Start: 3500mV, Run: 3150mV, Critical: 3100mV
`ch2`	101	`0x0065`	Semua = 0 (bench, batt tidak diperiksa)
`ch3`	102	`0x0066`	Start: 3500mV, Run: 3150mV, Critical: 3100mV
Gateway (root)	111	`0x006F`	—

Tabel Cache/Queue/Kapasitas CH

Komponen	Kapasitas	TTL / Timeout
NodeCache	32 slot	Stale: 300s, Expire: 3600s
AlarmQueue	8 slot	ACK timeout: 1500ms
MESH TX Queue	8 slot	—
Pending Downlink	16 slot	TTL: 1800000ms (30 menit)
Parent Candidates	8 slot	—

Perilaku CH_HELLO dan Parent Discovery

CH_HELLO payload (11 byte):

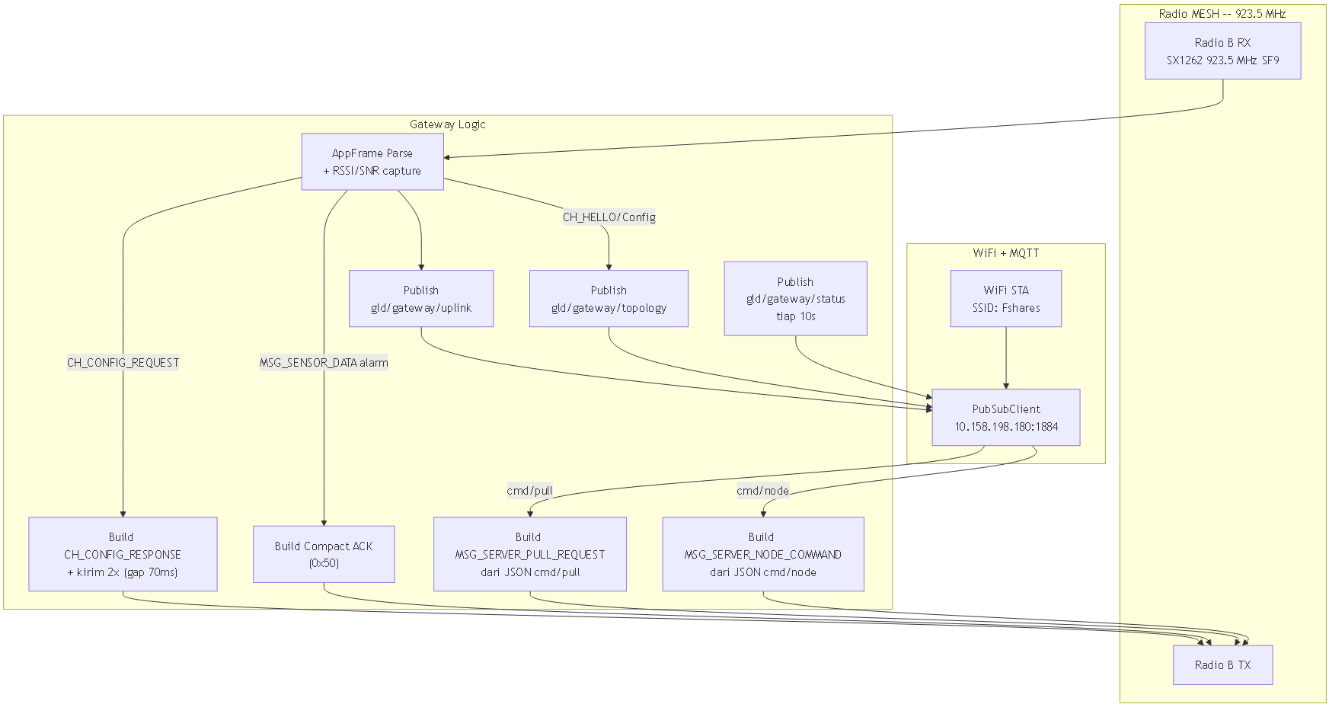
Offset	Field	Ukuran
0–1	CH ID	2
2–3	Parent ID	2
4–5	Battery mV	2
6–7	Uptime sec (low 16-bit)	2
8	Mesh depth	1
9–10	Alternate parent ID	2

Discovery timing:

Parameter	Nilai
Joining timeout	15000 ms
Config request interval	5000 ms
Response base delay	200 ms
Response slot gap	280 ms
Slot count	16
Jitter formula	`20 + (CH_ID & 0x000F) * 30 + (millis() % 40) ms`

6. Gateway

Diagram Internal Gateway



Gambar 10: Diagram internal Gateway — MESH RX/TX dan MQTT bridge.

Tabel MQTT Topics Gateway

Topic	Arah	Isi Payload
`gld/gateway/uplink`	GW → Server	JSON wrapper raw MESH frame (frameHex, rssi, snr, parse, typeFlags, ...)
`gld/gateway/topology`	GW → Server	JSON topology dari CH_HELLO, CH_CONFIG_REQUEST/RESPONSE
`gld/gateway/status`	GW → Server	JSON status: gatewayId, wifi, mqtt, meshReady, ip
`gld/gateway/cmd/#`	Server → GW	Wildcard subscription
`gld/gateway/cmd/pull`	Server → GW	JSON pull command: requestId + hopList
`gld/gateway/cmd/node`	Server → GW	JSON node command: cluster, node, id, ttl, hex

---

Tabel Config WiFi/MQTT Gateway

Parameter	Nilai
WiFi SSID	`Fshares`
WiFi Password	`kayabiasa`
MQTT Host	`10.158.198.180`
MQTT Port	`1884`
MQTT User	`deviot`
MQTT Pass	`deviot`
Gateway ID	`0x006F`

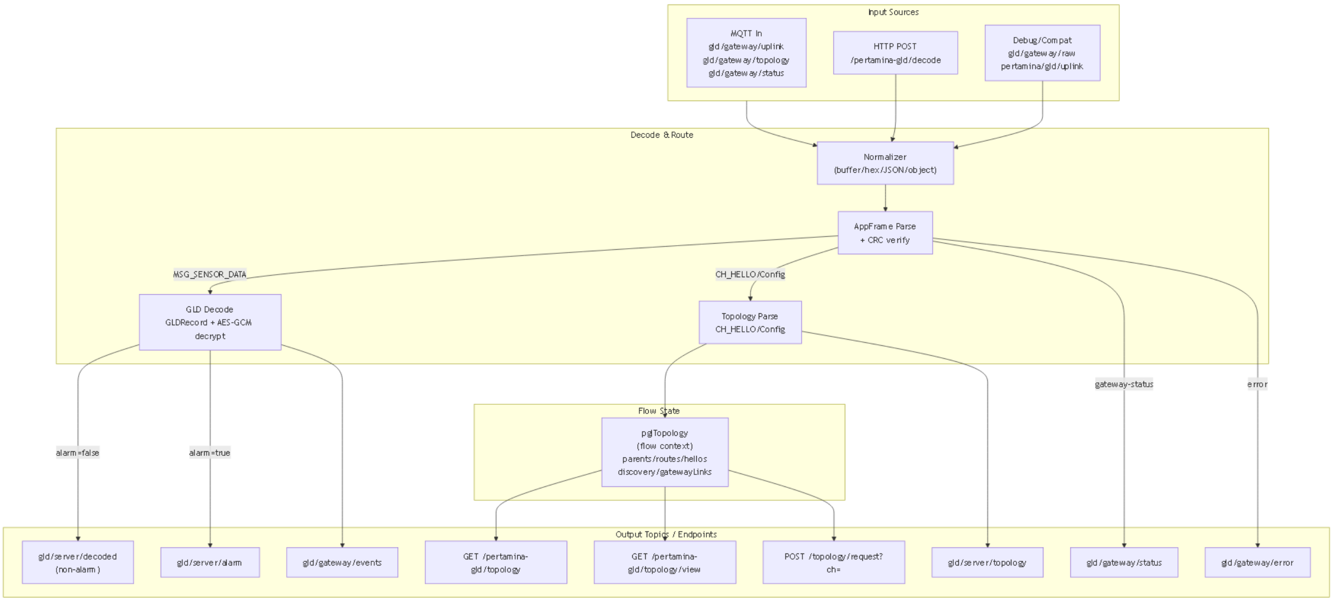
Parameter	Nilai
Topic Root	`gld/gateway`
MQTT Client ID	`pgl-gateway-006F-<MAC>`
WiFi Retry	5000 ms
MQTT Retry	3000 ms
Status Interval	10000 ms
Config Response Repeat	2x, gap 70 ms

Gateway Uplink JSON Fields

Field	Kapan Ada
`source = "gateway"`	Selalu
`gatewayId`	Selalu
`frameHex`	Selalu
`frameLen`	Selalu
`rssi, snr`	Selalu
`parseStatus`	Selalu
`typeFlags, msgType, srcId, dstId, seq, payloadLen`	AppFrame parse OK
`topology (object embed)`	Parse OK + MSG_CH_HELLO payload ≥8 byte

7. Server / Node-RED

Diagram Node-RED Logical Flow



Gambar 11: Logical flow Node-RED — input, decode, routing, dan output.

Tabel HTTP Endpoints Node-RED

Endpoint	Method	Fungsi
/pertamina-gld/decode	POST	Manual decode frame hex
/pertamina-gld/topology	GET	JSON topology: nodes, edges, routes, discovery
/pertamina-gld/topology/view	GET	HTML UI topology (auto-refresh 1000ms)
/pertamina-gld/topology/reset	POST	Reset state: parents, discovery, hellos, routes
/pertamina-gld/topology/request?ch=<id>	POST	Publish pull request ke cmd/pull jika route ada
/pertamina-gld/topology/delete?ch=<id>	POST	Hapus CH dari semua map topology

Tabel Topology State (pglTopology)

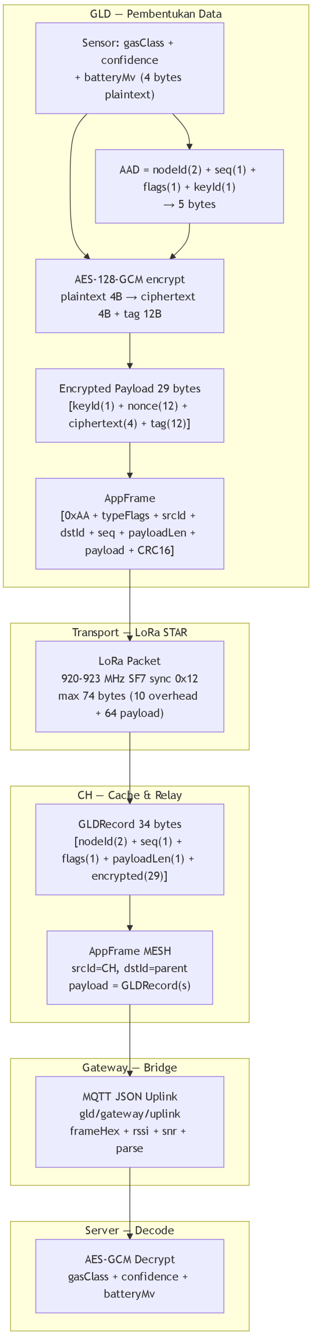
Field	Isi	TTL Default
parents	Installed CH parent (dari CH_HELLO)	900000 ms
discovery	CH_CONFIG discovery candidates	420000 ms
gatewayLinks	Gateway-heard RSSI/SNR dari CH_CONFIG	420000 ms
hellos	Latest CH_HELLO per CH	—
routes	Computed hop list Gateway→CH	—

Decode Output Fields (GLD Event)

Field	Keterangan	
ok	true jika decode berhasil	
kind	gld-event	
receivedAt	ISO timestamp	
nodeId, nodeIdHex	GLD ID	
seq	GLD record sequence	
flags	GLDRecord flags byte	
alarm	bit 0x01 dari flags	
externalPower	bit 0x10 dari flags	
testDevice	node ID dalam range 0xF000..0xFEFF	
payloadLen, payloadHex	encrypted payload	
dedupKey	`<cluster>:<node>:<seq>:<alarm\	normal>`
decryptOk	AES-GCM decrypt berhasil	
gasClass, gasName	hasil decrypt	
confidence	0–100	
batteryMv	mV baterai, 0xFFFF = invalid	

8. Protocol Visual Map

Diagram Lapisan Protocol



Gambar 12: Lapisan protocol dari data sensor hingga dekripsi di server.

Tabel AppFrame Field

Offset	Field	Ukuran	Keterangan
0	`Magic 0xAA`	1	Identifikasi frame

Offset	Field	Ukuran	Keterangan
1	typeFlags	1	Type (6-bit) + Flags (2-bit)
2–3	srcId	2	Source node ID (big-endian)
4–5	dstId	2	Destination node ID (big-endian)
6	seq	1	Sender sequence counter
7	payloadLen	1	Panjang payload (0–64 STAR, 0–80 MESH)
8..	payload	N	Isi sesuai message type
akhir	CRC16-CCITT-FALSE	2	Over header + payload

Tabel Message Types

Hex	Nama	Penggunaan
`0x10`	`MSG_SENSOR_DATA`	GLD uplink, alarm push, recovery clear, compact ACK reuse
`0x14`	`MSG_NODE_DOWNLINK`	CH → GLD downlink command
`0x30`	`MSG_SERVER_PULL_REQUEST`	Server → GW → CH: pull data request
`0x31`	`MSG_CLUSTER_DATA_RESPONSE`	CH → GW: pull data response
`0x32`	`MSG_SERVER_NODE_COMMAND`	GW → CH: command ke GLD tertentu
`0x33`	`MSG_CH_HELLO`	CH topology/liveness announcement
`0x34`	`MSG_CH_CONFIG_REQUEST`	CH discovery broadcast
`0x35`	`MSG_CH_CONFIG_RESPONSE`	CH/GW balasan discovery

Tabel typeFlags GLD

Name	Nilai	Kondisi
Normal + Battery	`0x10`	gasClass=0 atau conf<30, baterai
Normal + External	`0x90`	gasClass=0 atau conf<30, external power
Alarm + Battery	`0x50`	alarm aktif, baterai
Alarm + External	`0xD0`	alarm aktif, external power
Compact Alarm ACK	`0x50`	ACK dari CH/GW ke pengirim alarm

Diagram GLD Payload — 4 Byte Plaintext dan 29 Byte Encrypted

```
PLAINTEXT (4 bytes):
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■  gasClass   ■  confidence  ■        batteryMv (BE)         ■
■   1 byte    ■    1 byte    ■            2 bytes            ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

ENCRYPTED PAYLOAD (29 bytes):
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■  keyId  ■            nonce            ■  ciphertext  ■            tag            ■
```

```
■  1 byte  ■         12 bytes          ■  4 bytes   ■         12 bytes              ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

AAD (5 bytes, tidak dikirim tapi dipakai untuk enkripsi/validasi):
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■    nodeId    ■    seq    ■    flags    ■  keyId   ■
■   2 bytes    ■  1 byte   ■   1 byte    ■  1 byte  ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

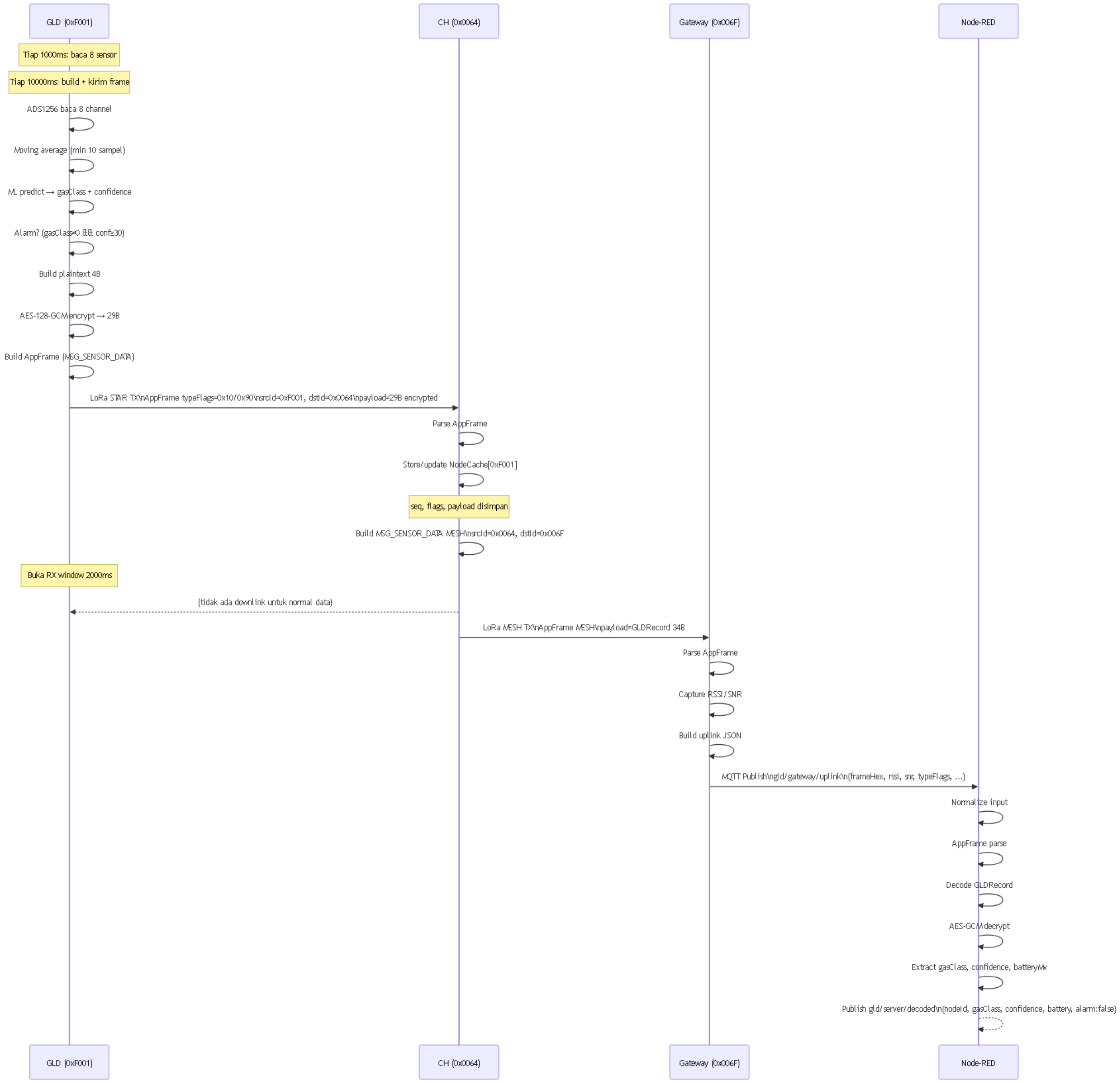
Diagram GLDRecord (34 byte) — Unit Cache dan Respons CH

```
GLDRecord (34 bytes):
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■    nodeId    ■    seq    ■    flags    ■  payloadLen  ■   encrypted payload (29 bytes)     ■
■   2 bytes    ■  1 byte   ■   1 byte    ■    1 byte    ■                                    ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 offset 0-1       offset 2   offset 3       offset 4         offset 5-33

Record flags:
  0x01 = alarm
  0x10 = external power
```

9. Normal Data Flow

Sequence Diagram Normal Data Flow



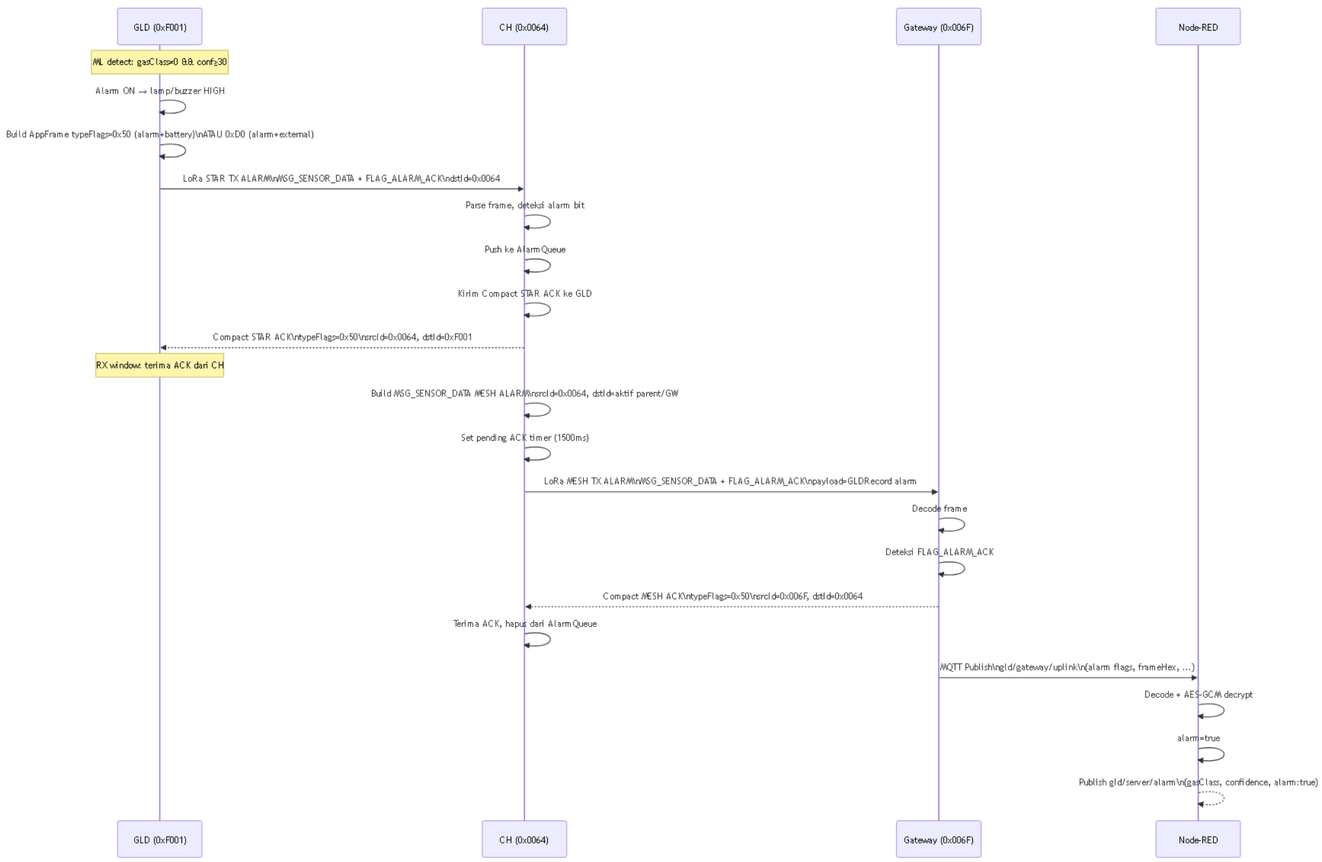
Gambar 13: Sequence diagram alur data normal GLD → CH → Gateway → Node-RED.

Tabel Input/Output Per Tahap — Normal Flow

Tahap	Input	Proses	Output
GLD Scan	Tegangan ADS1256 × 8	Moving avg + ML predict	gasClass, confidence, batteryMv
GLD Encode	4-byte plaintext	AES-128-GCM + AppFrame	29-byte encrypted dalam frame
CH Cache	AppFrame STAR	Parse + NodeCache update	GLDRecord dalam cache
CH MESH TX	NodeCache entry	Build MSG_SENSOR_DATA MESH	AppFrame MESH ke GW
GW Receive	AppFrame MESH	Parse + RSSI/SNR	JSON uplink
GW Publish	JSON uplink	PubSubClient.publish	`MQTT gld/gateway/uplink`
NR Decode	JSON dengan frameHex	AppFrame + GLD + AES-GCM	`gld/server/decoded`

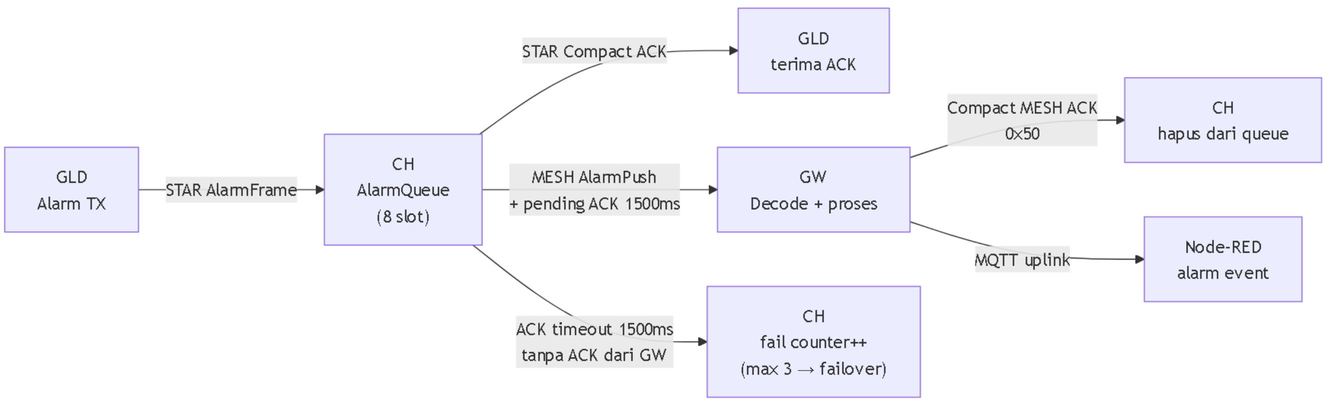
10. Alarm Flow

Sequence Diagram Alarm



Gambar 14: Sequence diagram alarm flow — dari deteksi GLD hingga server.

Diagram Queue/ACK Alarm



Gambar 15: Mekanisme queue dan ACK alarm CH.

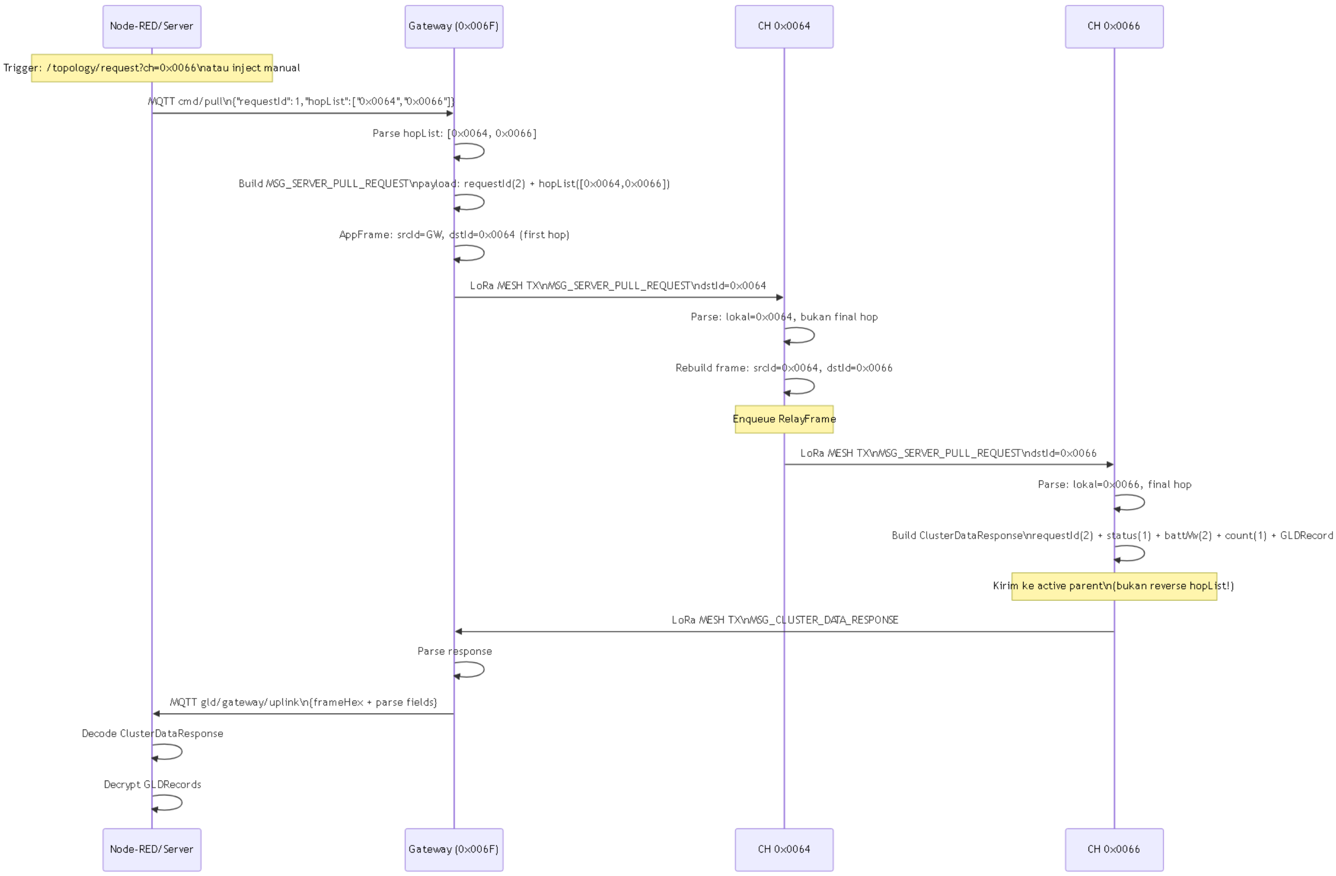
Tabel Alarm Flags

Flag	Nilai	Penggunaan
`FLAG_ALARM_ACK`	`0x40`	Bit alarm dalam typeFlags
`FLAG_GLD_EXT_POWER`	`0x80`	Bit external power dalam typeFlags
`NC_FLAG_ALARM`	`0x01`	Flag alarm dalam GLDRecord/NodeCache
`NC_FLAG_EXT_POWER`	`0x10`	Flag external power dalam GLDRecord

> ■ ***Caveat:*** *Saat ini* `checkAlarmAckTimeout()` *CH hanya log timeout, hapus item dari AlarmQueue, dan increment counter —* ***tidak retry*** *alarm frame yang sama. Parent failover terpicu jika counter mencapai threshold 3, dan RECOVERY (restart ESP) terpicu jika no-ACK burst mencapai 5.*

11. Server Pull Flow

Sequence Diagram Server Pull



Gambar 16: Sequence diagram server pull flow — hopList routing hingga respons.

Diagram HopList

```
Server kirim: hopList = [0x0064, 0x0066]

Step 1: GW → CH 0x0064 (first hop)
  AppFrame: srcId=GW(0x006F), dstId=0x0064
```

```
  Payload: requestId + [0x0064, 0x0066]

Step 2: CH 0x0064 relay → CH 0x0066
  CH 0x0064 baca payload, cari posisi lokal (index 0)
  bukan final hop → relay ke next: dstId=0x0066
  AppFrame: srcId=0x0064, dstId=0x0066
  Payload: requestId + [0x0064, 0x0066] (sama)

Step 3: CH 0x0066 final → kirim respons ke active parent
  Final hop → build ClusterDataResponse
  Kirim ke runtimeConfig.meshDstId (parent aktif, bukan reverse)
```

Tabel Data Status Respons Pull

Nilai	Nama	Arti
`0x00`	`DataOk`	Data tersedia dan valid
`0x01`	`DataEmpty`	Cache CH kosong, tidak ada GLD
`0x02`	`DataNotAvail`	Data tidak tersedia (belum ada data)
`0x03`	`DataStale`	Data ada tapi terlalu lama (>300s)
`0x04`	`DataBusy`	CH sedang sibuk
`0x05`	`DataInvalid`	Data tidak valid

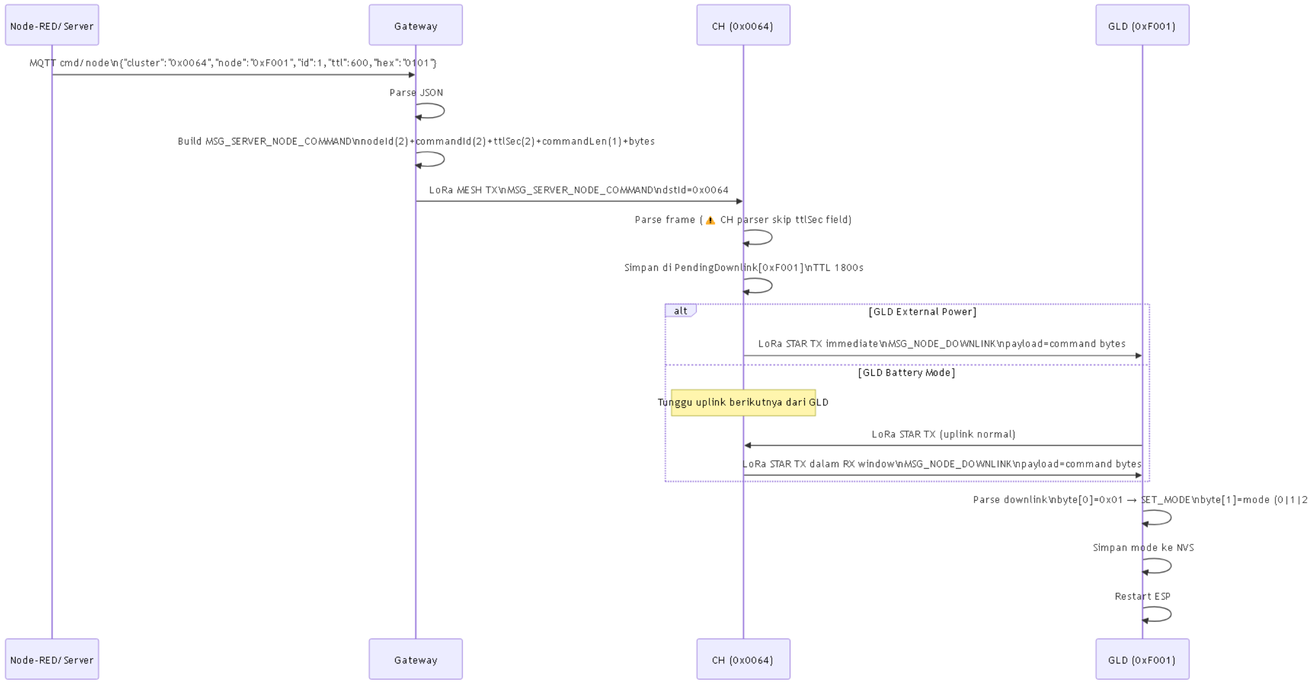
ClusterDataResponse Payload

Offset	Field	Ukuran
0–1	requestId	2
2	data status	1
3–4	CH battery mV	2
5	record count	1
6..	GLDRecords (34 byte per record)	variable

■ ***Kapasitas:*** *MESH payload max 80 byte. Header 6 byte + 2 × 34 byte = 74 byte. Maksimal* ***2 GLDRecord*** *per respons.*

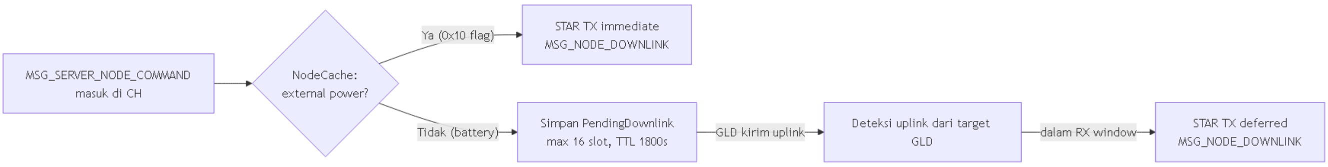
12. Downlink Command Flow

Sequence Diagram Downlink



Gambar 17: Sequence diagram downlink command flow.

Immediate vs Deferred Downlink



Gambar 18: Immediate downlink (external power) vs deferred downlink (battery mode).

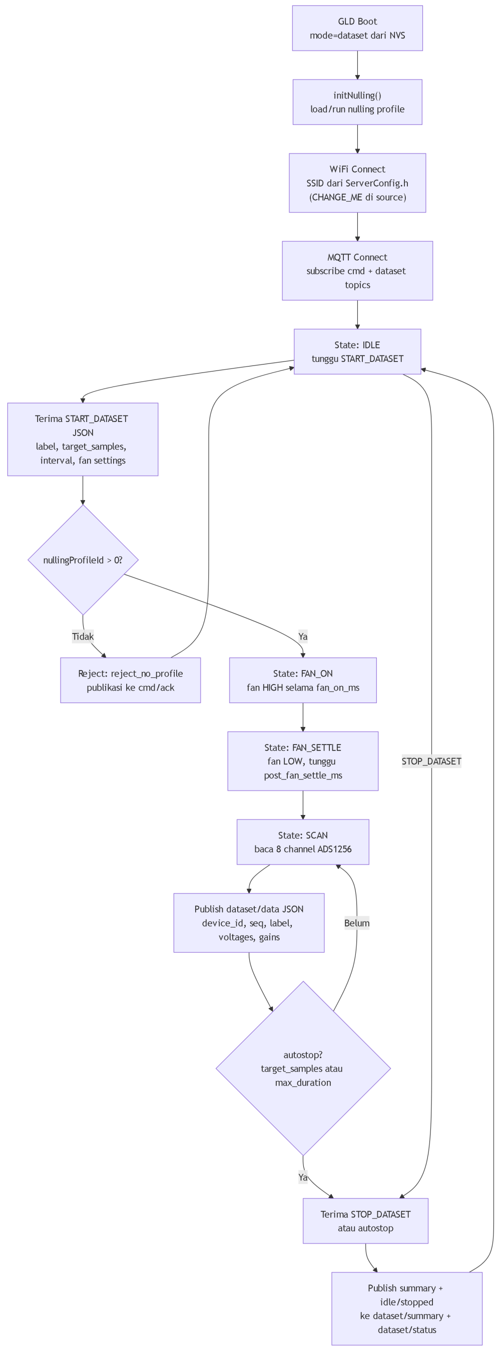
GLD Downlink Parser

Byte	Field	Nilai Saat Ini
0	Command type	`0x01 = SET_MODE`
1	Mode value	`0 = inference, 1 = dataset, 2 = nulling`

■ ***Caveat TTL Mismatch:*** *Gateway build payload* `nodeId(2) + commandId(2) + ttlSec(2) + commandLen(1) + commandBytes`*, sedangkan CH parser membaca* `nodeId(2) + commandId(2) + commandLen(1) + commandBytes`*. Field* `ttlSec` *dari Gateway dilewati CH dan menimpa* `commandLen`*, sehingga perintah downlink saat ini* ***tidak ter-align****. Ini adalah mismatch yang ada di source saat ini.*

13. Dataset Mode Flow

Flowchart Dataset Mode



Gambar 19: Flowchart dataset mode — inisialisasi, capture loop, dan autostop.

Tabel MQTT Topics Dataset

Topic	Arah	Isi
`gas-leak-detector/F001/cmd`	Server → GLD	`{"cmd":"SET_MODE","mode":"..."}`
`gas-leak-detector/F001/dataset`	Server → GLD	`{"cmd":"START_DATASET" atau "STOP_DATASET", ...}`
`gas-leak-detector/F001/dataset/data`	GLD → Server	JSON data tiap scan
`gas-leak-detector/F001/dataset/status`	GLD → Server	Status idle/running/stopped

Topic	Arah	Isi
`gas-leak-detector/F001/dataset/summary`	GLD → Server	Ringkasan sesi dataset
`gas-leak-detector/F001/cmd/ack`	GLD → Server	Ack command (termasuk reject)

Tabel Field JSON Dataset Data

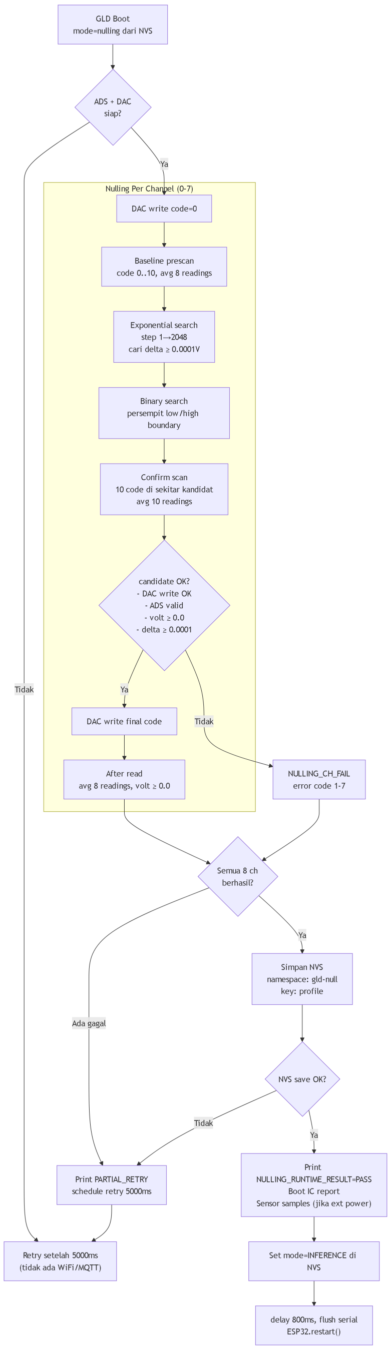
Field	Keterangan
`device_id`	`"F001"`
`node_id`	`0xF001 numerik`
`mode`	`"DATASET"`
`seq`	Sequence dataset
`timestamp_ms`	`millis()`
`label`	Label dari START_DATASET
`nulling_profile_id`	Profile ID yang dipakai
`sensor_voltage`	Array 8 tegangan (float)
`sensor_gain`	Array 8 gain/status dari pembacaan
`feature_order`	Nama sensor dalam urutan hardware

Parameter START_DATASET

Field	Default	Keterangan
`label`	`"unknown"`	Label untuk data capture
`target_samples`	0	0 = infinite
`sample_interval_ms`	1000	Interval antar scan
`max_duration_ms`	0	0 = infinite
`use_fan_intake`	true	Gunakan kipas intake
`fan_on_ms`	1000	Durasi kipas menyala
`post_fan_settle_ms`	0	Settle setelah kipas mati

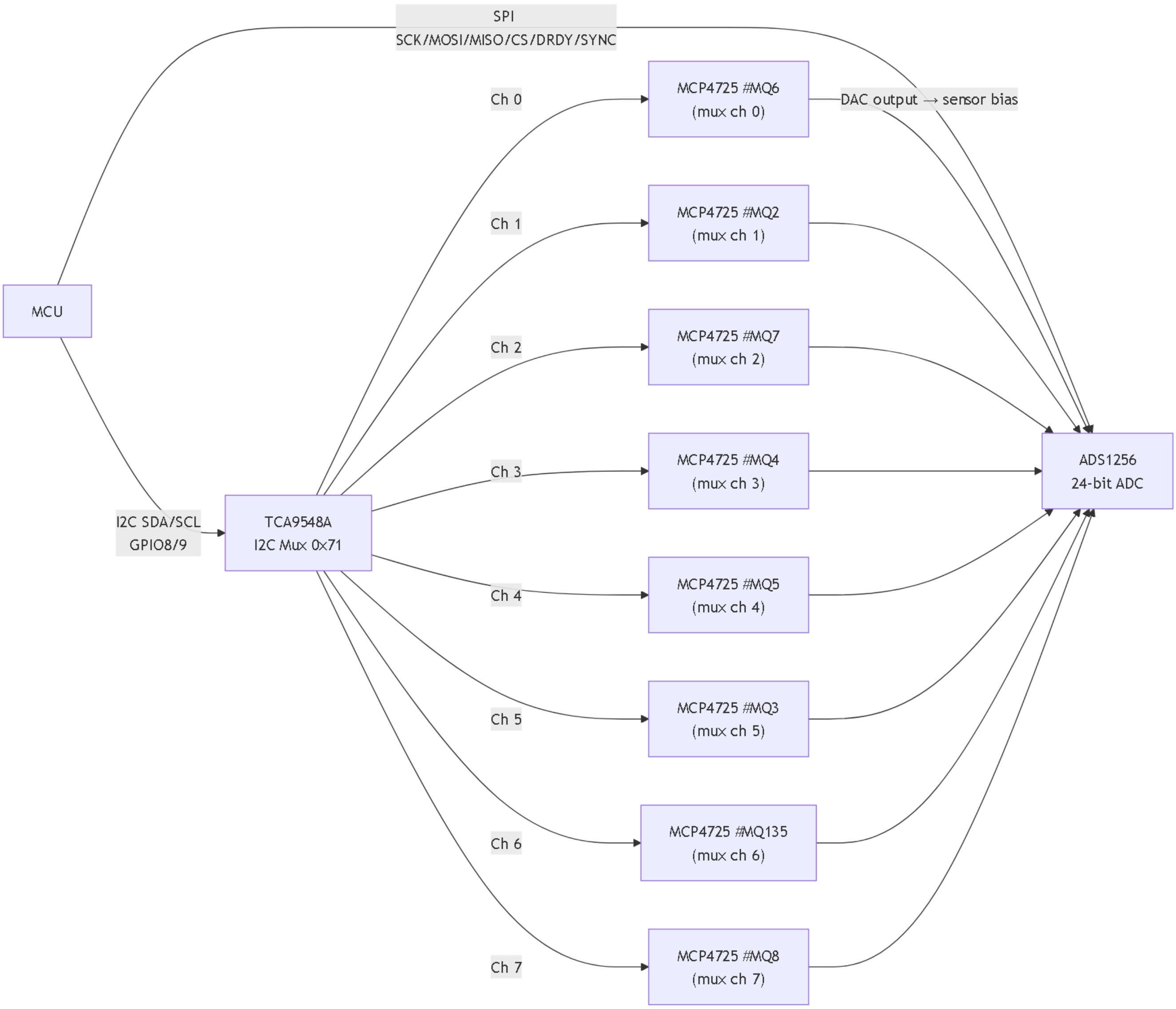
14. Nulling Mode Flow

Flowchart Nulling Mode



Gambar 20: Flowchart nulling mode — per-channel calibration loop.

Diagram Relasi DAC/MUX/ADS



Gambar 21: Relasi DAC (MCP4725 via TCA9548A) dan ADC (ADS1256) dalam nulling.

Tabel Threshold/Count/Timing Nulling

Konstanta	Nilai	Keterangan
Average count	8	Jumlah sampel per rata-rata
Confirm count	10	Jumlah sampel per kandidat confirm
Baseline prescan max code	10	Kode DAC 0..10 untuk prescan
Exponential initial step	1	Step awal exponential search
Exponential max step	2048	Step maksimum sebelum berhenti
Confirm window	10 DAC codes	Area konfirmasi di sekitar kandidat
Settle delay	5 ms	Jeda setelah DAC write
Threshold	`0.0001 V`	Delta minimum yang diterima
Minimum final voltage	`0.0 V`	Tegangan akhir harus ≥ ini

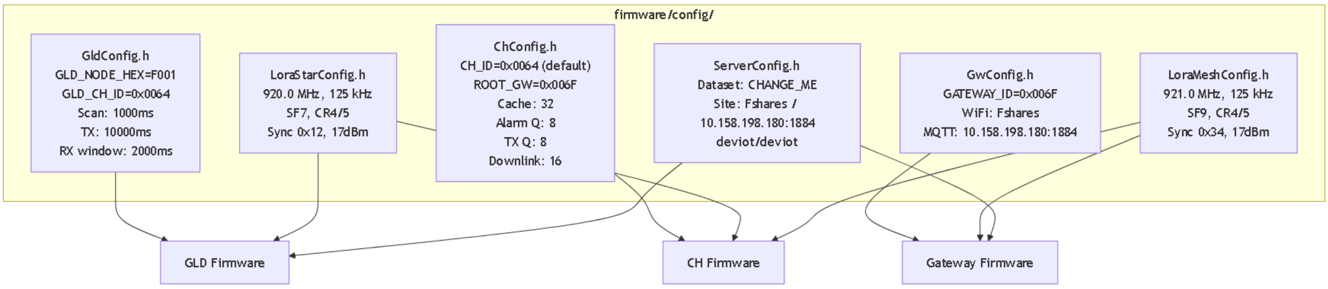
Nulling Error Codes

Code	Alasan Gagal
1	`dac_zero_write_failed`
2	`baseline_no_valid_samples`
3	`exponential_range_not_found`
4	`confirm_failed`
5	`dac_final_write_failed`
6	`after_read_invalid`
7	`after_voltage_negative`

■ ***Catatan:*** *Nulling mode berjalan* ***offline*** *— tidak ada WiFi, MQTT, atau LoRa. Setelah sukses, firmware otomatis mengubah mode ke* `inference` *dan restart ESP.*

15. Konfigurasi Penting

Visual Config Map Per File



Gambar 22: Peta config file dan dependency ke firmware.

Tabel Node IDs

Node	ID (hex)	ID (desimal)	Peran
GLD	`0xF001`	61441	Sensor gas node
CH1	`0x0064`	100	Cluster Head 1
CH2	`0x0065`	101	Cluster Head 2
CH3	`0x0066`	102	Cluster Head 3
Gateway	`0x006F`	111	Root Gateway/MESH
Broadcast	`0xFFFF`	65535	Broadcast ke semua

Tabel Radio Config STAR/MESH

Parameter	STAR (GLD↔CH)	MESH (CH↔GW)
Frekuensi	920.0 MHz	921.0 MHz

Parameter	STAR (GLD↔CH)	MESH (CH↔GW)
Bandwidth	125 kHz	125 kHz
Spreading Factor	SF7	SF9
Coding Rate	4/5	4/5
Sync Word	`0x12`	`0x34`
TX Power	17 dBm	17 dBm
Preamble	8	8
SPI Clock	2 MHz	2 MHz
TCXO	1.6V (fallback 0.0V)	1.6V (fallback 0.0V)
Max Payload	64 bytes	80 bytes

Tabel Timing Penting

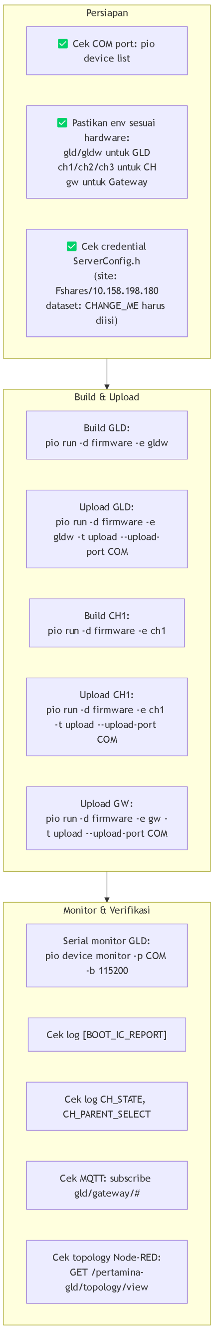
Parameter	Nilai	Komponen
GLD scan interval	1000 ms	GLD
GLD TX interval	10000 ms	GLD
GLD RX window	2000 ms	GLD
GLD WiFi timeout	15000 ms	GLD
GLD MQTT retry	3000 ms	GLD
CH joining timeout	15000 ms	CH
CH config request interval	5000 ms	CH
CH hello interval	300000 ms	CH
CH alarm ACK timeout	1500 ms	CH
CH pending downlink TTL	1800000 ms	CH
CH parent health timeout	180000 ms	CH
CH parent min dwell	300000 ms	CH
CH housekeeping	60000 ms	CH
CH route verify	600000 ms	CH
GW status interval	10000 ms	Gateway
GW WiFi retry	5000 ms	Gateway
GW MQTT retry	3000 ms	Gateway

Tabel Battery Thresholds

Threshold	env:ch1/ch3	env:ch2	Kondisi
Start (BATT_START_MV)	3500 mV	0 (disabled)	Harus terpenuhi 8x berturut sebelum radio init
Run Min (BATT_RUN_MIN_MV)	3150 mV	0 (disabled)	Di bawah ini → LOW_POWER state
Critical (BATT_CRITICAL_MV)	3100 mV	0 (disabled)	TX diblokir di LOW_POWER

16. Operasi Lapangan

Checklist Operasi Visual



Gambar 23: Checklist operasi lapangan.

Command Build/Upload

```
# Cek COM port
pio device list

# Build GLD (board WROOM, env gldw)
pio run -d firmware -e gldw
```

```
# Upload GLD ke COM9 (contoh)
pio run -d firmware -e gldw -t upload --upload-port COM9

# Build + Upload CH1
pio run -d firmware -e ch1 -t upload --upload-port COM3

# Build + Upload CH2
pio run -d firmware -e ch2 -t upload --upload-port COM5

# Build + Upload Gateway
pio run -d firmware -e gw -t upload --upload-port COM38

# Serial monitor GLD
pio device monitor -p COM9 -b 115200

# Serial monitor CH
pio device monitor -p COM3 -b 115200
```

■ ***Catatan:*** *COM port di atas adalah contoh dari sesi bench sebelumnya. Selalu cek ulang dengan* `pio device list` *sebelum upload.*

Cara Membaca Log Serial GLD

Log Token	Arti
`[BOOT_IC_REPORT]`	Tabel IC check setelah boot
`[BOOT_SENSOR_SAMPLES]`	5 baris sampel sensor (external power)
`MODE_READY`	Mode siap dieksekusi
`NULLING_SERVICE_START`	Nulling mulai
`NULLING_CH_OK / NULLING_CH_FAIL`	Per-channel nulling result
`NULLING_RUNTIME_RESULT=PASS`	Semua channel sukses
`PARTIAL_RETRY`	Ada channel gagal, retry

Cara Membaca Log Serial CH

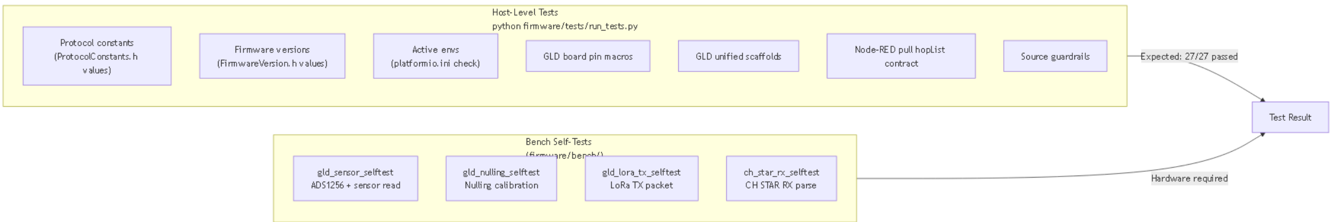
Log Token	Arti
`CH_STATE`	Transisi state machine
`CH_PARENT_SELECT`	Parent terpilih
`CH_STAR_RX`	Terima frame dari GLD
`CH_STAR_PROCESS`	Proses + update cache
`CH_MESH_TX_KIND`	Frame MESH yang dikirim
`CH_ALARM_ACK_RECV`	Terima ACK alarm
`CH_ALARM_ACK_TIMEOUT`	Timeout ACK alarm
`CH_HELLO_TX`	Kirim CH_HELLO
`CH_CACHE_SUMMARY`	Ringkasan isi NodeCache

Cara Membaca Log Serial Gateway

Log Token	Arti
`parseStatus di MQTT JSON`	Decode result frame MESH
`rssi, snr di MQTT JSON`	Kualitas link MESH
`frameHex`	Raw frame untuk debugging
`topology object`	CH_HELLO embedded

17. Verifikasi dan Test

Diagram Test Coverage



Gambar 24: Coverage test — host-level dan bench self-test.

Tabel Test Command

Command	Platform	Expected
`python firmware/tests/run_tests.py`	Host (Python)	`27/27 tests passed`
`pio run -d firmware/bench -e gld_sensor_selftest_esp32s3`	ESP32-S3 bench	Sensor read OK
`pio run -d firmware/bench -e gld_nulling_selftest_esp32s3`	ESP32-S3 bench	Nulling OK
`pio run -d firmware/bench -e gld_lora_tx_selftest_esp32s3`	ESP32-S3 bench	LoRa TX OK
`pio run -d firmware/bench -e ch_star_rx_selftest_esp32s3`	ESP32-S3 bench	STAR RX parse OK

Tabel Status Test (Yang Boleh Diklaim)

Area	Status
Protocol constants (source)	■ Terimplementasi, ter-guard test host
Firmware versions (source)	■ Terimplementasi, ter-guard test host
Active env config (platformio.ini)	■ Ter-guard test host
GLD board pin macros	■ Ter-guard test host
GLD unified scaffolds	■ Ter-guard test host
Node-RED pull hopList contract	■ Ter-guard test host
GLD sensor read	Source ada, bench test terpisah

Area	Status
Nulling calibration	Source ada, bench test terpisah
LoRa STAR TX	Source ada, bench test terpisah
CH STAR RX parse	Source ada, bench test terpisah
End-to-end GLD→Server dengan AES-GCM	Source + Node-RED ada, **belum ada test otomatis E2E**
Multi-hop pull via hopList	Source ada, **belum ada test integrasi otomatis**
Downlink command (TTL mismatch)	Source ada caveat, **tidak ter-guard test**

18. Known Caveats / Risiko

Area	Caveat	Dampak	Status di Source	Rekomendasi
Downlink TTL mismatch	`Gateway build nodeId+commandId+ttlSec+commandLen+bytes, CH parser skip ttlSec sehingga commandLen salah offset`	Downlink command dari server ke GLD **tidak berfungsi** via remote	`Terdokumentasi eksplisit di ch-gw/final_design.md dan gld-ch/payload-contract.draft.md`	Align satu sisi (tambah ttlSec parsing di CH atau hapus dari GW) sebelum live deploy
Dataset WiFi credential	`PGL_SERVER_DATASET_WIFI_SSID/PASSWORD dan MQTT_HOST di ServerConfig.h bernilai "CHANGE_ME"`	Dataset mode tidak bisa konek WiFi/MQTT sampai credential diisi	`Terlihat di firmware/config/ServerConfig.h`	Isi dengan credential prod sebelum build dataset
Alarm ACK tidak retry	Saat ACK timeout (1500ms), CH hanya log dan hapus alarm dari queue — tidak kirim ulang	Alarm bisa hilang jika link MESH sementara drop	`Terdokumentasi di ch/final_design.md`	Implementasikan alarm retry logic jika reliabilitas diutamakan
Dataset flow JSON mismatch	`pertamina-gld-dataset.flow.json expect field ch, gain, ok, nodeId, ts_ms, profileId — sedangkan GLD unified kirim sensor_voltage, sensor_gain, node_id, timestamp_ms, nulling_profile_id`	Dataset recorder dari flow lama tidak kompatibel dengan GLD unified	`Terdokumentasi di server/final_design.md`	`Gunakan gld_dataset_recorder.py yang cocok, atau update flow dataset`
MQTT buffer limit	GLD dan GW MQTT buffer 1024 bytes	Payload besar (dataset JSON + 8 voltage) mendekati limit	`Source: GLD_MQTT_BUFFER_SIZE = 1024`	Verifikasi ukuran payload dataset di lapangan
NodeCache expire	Entry GLD expire setelah 3600s tanpa data	CH tidak kirim data GLD yang lama ke server sampai GLD kirim lagi	`Source: CACHE_EXPIRE_MS = 3600000`	Normal behavior, dokumentasikan ke operator
Gateway parent di CH	CH mem-boot ulang fresh setiap restart dan Gateway link harus fresh (NVS Gateway parent di-clear saat boot)	Perlu waktu re-discovery setelah restart CH	`Source: ch/final_design.md`	Biarkan joining timeout 15s untuk stabilisasi

Area	Caveat	Dampak	Status di Source	Rekomendasi
Default test AES key	`Node-RED default key 000102030405060708090A0B0C0D0E0F via env GLD_AES128_KEY_HEX`	Jika env tidak diset, server memakai test key yang sama dengan GLD	`Source: server/final_design.md`	`Set GLD_AES128_KEY_HEX env di prod Node-RED dan GLD`

19. Appendix

A. Visual Glossary

Term	Definisi
AppFrame	Format frame biner sistem: magic + typeFlags + srcId + dstId + seq + payloadLen + payload + CRC16
AES-128-GCM	Enkripsi authenticated dengan key 128-bit, nonce 12 byte, tag 12 byte
AAD	Additional Authenticated Data — 5 byte untuk validasi enkripsi tanpa decrypt
CH	Cluster Head — node relay antara GLD dan Gateway
GLD	Gas Leak Detector — node sensor gas utama
STAR	Topologi radio GLD↔CH — satu GLD ke satu CH
MESH	Topologi radio CH↔CH↔Gateway — multi-hop backbone
NodeCache	Cache GLD record di CH — 32 slot, stale 300s, expire 3600s
AlarmQueue	Queue alarm GLD pending ACK di CH — 8 slot, ACK timeout 1500ms
GLDRecord	Unit data GLD 34 byte: nodeId+seq+flags+payloadLen+encrypted
hopList	Daftar CH yang harus dilalui pull request dalam urutan tertentu
typeFlags	Byte kombinasi message type (6-bit) + FLAG_ALARM_ACK + FLAG_GLD_EXT_POWER
pglTopology	Flow context Node-RED untuk menyimpan state jaringan CH/GW
NVS	Non-Volatile Storage ESP32 — penyimpanan mode, nulling profile, parent ID
TCXO	Temperature-Compensated Crystal Oscillator — sumber clock akurat SX1262

B. Daftar File Penting

File	Peran
`firmware/platformio.ini`	Definisi env build runtime
`firmware/shared/include/ProtocolConstants.h`	Semua konstanta protokol
`firmware/shared/include/FirmwareVersion.h`	Versi firmware semua komponen
`firmware/config/GldConfig.h`	Config GLD node
`firmware/config/ChConfig.h`	Config CH node
`firmware/config/GwConfig.h`	Config Gateway

File	Peran
`firmware/config/ServerConfig.h`	Config WiFi/MQTT server
`firmware/config/LoraStarConfig.h`	Config radio STAR
`firmware/config/LoraMeshConfig.h`	Config radio MESH
`firmware/gld/src/GldUnifiedMain.cpp`	Main runtime GLD
`firmware/gld/src/GldNullingService.cpp`	Nulling calibration
`firmware/ch/src/ChStarMeshRuntimeMain.cpp`	Main runtime CH
`firmware/ch/src/ChRuntime.cpp`	State machine CH
`firmware/gateway/src/GatewayMqttMeshMain.cpp`	Main runtime Gateway
`server/nodered/functions/pertamina-gld-decode.js`	Decode + AES-GCM server
`server/nodered/apply-pertamina-gld-flow.js`	Generator Node-RED flow
`server/nodered/pertamina-gld-server.flow.json`	Snapshot flow server

C. Daftar Env Aktif

Env	Board	Firmware	Tujuan
`gld`	4D-ESP32S3-Gen4	`GLD 0.8.12`	GLD runtime standar
`gldw`	ESP32-S3-WROOM-1U-N16R8	`GLD 0.8.12`	GLD runtime WROOM 16MB
`ch1`	4D-ESP32S3-Gen4	`CH 0.7.1`	`CH ID 0x0064`
`ch2`	4D-ESP32S3-Gen4	`CH 0.7.1`	`CH ID 0x0065 (batt disabled)`
`ch3`	4D-ESP32S3-Gen4	`CH 0.7.1`	`CH ID 0x0066`
`gw`	4D-ESP32S3-Gen4	`GW 0.1.3`	Gateway

D. Daftar MQTT Topics

Topic	Arah	Komponen
`gld/gateway/uplink`	GW → Server	Uplink frame MESH wrapped JSON
`gld/gateway/topology`	GW → Server	Topology CH_HELLO/Config JSON
`gld/gateway/status`	GW → Server	Status GW periodic
`gld/gateway/cmd/#`	Server → GW	Command wildcard
`gld/gateway/cmd/pull`	Server → GW	Pull request JSON
`gld/gateway/cmd/node`	Server → GW	Node command JSON
`gld/server/decoded`	Node-RED internal	Decoded normal GLD event
`gld/server/alarm`	Node-RED internal	Decoded alarm event
`gld/server/topology`	Node-RED internal	Topology event
`gld/gateway/error`	Node-RED internal	Error event
`gas-leak-detector/F001/cmd`	Server → GLD	Command dataset mode
`gas-leak-detector/F001/dataset`	Server → GLD	Dataset start/stop
`gas-leak-detector/F001/dataset/data`	GLD → Server	Data scan dataset

Topic	Arah	Komponen
`gas-leak-detector/F001/dataset/status`	GLD → Server	Status dataset
`gas-leak-detector/F001/dataset/summary`	GLD → Server	Summary sesi
`gas-leak-detector/F001/cmd/ack`	GLD → Server	Command ack

E. Daftar Message Types

Hex	Constant	Penggunaan
`0x10`	`MSG_SENSOR_DATA`	GLD uplink, alarm push, recovery, compact ACK
`0x14`	`MSG_NODE_DOWNLINK`	CH → GLD downlink
`0x30`	`MSG_SERVER_PULL_REQUEST`	Server pull request
`0x31`	`MSG_CLUSTER_DATA_RESPONSE`	CH pull response
`0x32`	`MSG_SERVER_NODE_COMMAND`	Server command ke GLD via CH
`0x33`	`MSG_CH_HELLO`	CH topology hello
`0x34`	`MSG_CH_CONFIG_REQUEST`	CH parent discovery
`0x35`	`MSG_CH_CONFIG_RESPONSE`	CH/GW discovery response

F. Daftar Log Token Penting

Token	Komponen	Arti
`[BOOT_IC_REPORT]`	GLD	Tabel IC check boot
`[BOOT_SENSOR_SAMPLES]`	GLD	5 baris sampel sensor
`MODE_READY`	GLD	Mode siap
`NULLING_SERVICE_START`	GLD	Nulling dimulai
`NULLING_CH_OK / NULLING_CH_FAIL`	GLD	Per-channel nulling result
`NULLING_RUNTIME_RESULT=PASS`	GLD	Semua channel sukses
`PARTIAL_RETRY`	GLD	Ada gagal, retry
`CH_STATE`	CH	State machine transition
`CH_PARENT_SELECT`	CH	Parent terpilih
`CH_STAR_RX`	CH	Terima dari GLD
`CH_ALARM_ACK_RECV`	CH	ACK alarm diterima
`CH_ALARM_ACK_TIMEOUT`	CH	ACK alarm timeout
`CH_HELLO_TX`	CH	Kirim topology hello
`CH_CACHE_SUMMARY`	CH	Ringkasan cache
`CH_MESH_TX_KIND`	CH	Jenis frame MESH dikirim

Dokumen ini dibuat berdasarkan source code dan konfigurasi repo Pertamina GLD per tanggal 2026-06-29.

Source priority: `firmware/platformio.ini` → `shared/include/*.h` → `config/*.h` → `gld/`, `ch/`, `gateway/` → `server/nodered/` → `docs/design/*/final_design.md`